

William Anderson

Senior Software Engineer

Seminole, FL | williamandresondev@outlook.com | +1 (407) 801-1287 |
<https://www.linkedin.com/in/william-andreson-39b88b3aa>

Professional Summary

Senior **Node.js** Software Engineer with 12+ years of experience building scalable SaaS, fintech, healthcare, logistics, and cloud-based platforms using **Node.js 14.x–20.x**, **TypeScript 4.x–5.x**, **Express.js 4.x**, **NestJS 8.x–10.x**, **PostgreSQL**, MongoDB, Redis, Docker, Kubernetes, and AWS. Strong background in API architecture, microservices, event-driven systems, performance tuning, legacy migration, CI/CD automation, and production bug resolution, with proven delivery of secure backend systems that improved reliability, reduced latency, and supported high-volume business workflows.

Technical Skills

- **Backend:** **Node.js 14.x–20.x**, **TypeScript 4.x–5.x**, JavaScript ES6+, Express.js 4.x, NestJS 8.x–10.x, Fastify, REST API, GraphQL, WebSocket
- **Database:** PostgreSQL 12.x–16.x, MySQL 8.x, MongoDB 4.x–7.x, Redis 6.x–7.x, DynamoDB, Prisma, TypeORM, Sequelize
- **Cloud & DevOps:** AWS, Lambda, ECS, S3, CloudWatch, SNS, SQS, Docker, Kubernetes, GitHub Actions, GitLab CI, Terraform
- **Architecture:** Microservices, Event-Driven Architecture, CQRS, API Gateway, Serverless, Message Queues, Domain-Driven Design
- **Testing & Quality:** Jest, Mocha, Chai, Supertest, Cypress, ESLint, SonarQube, OpenAPI/Swagger
- **Monitoring:** Datadog, Prometheus, Grafana, OpenTelemetry, CloudWatch Logs, Sentry
- **Frontend:** React 17.x–18.x, Next.js 12.x–14.x, Redux Toolkit, Tailwind CSS, Material UI

Professional Experience

Viaro Networks Inc.

Senior Software Engineer | January 2021 – December 2025

- Built enterprise SaaS and fintech workflow platforms using **Node.js 16.x–20.x**, **TypeScript 4.x–5.x**, **NestJS 8.x–10.x**, PostgreSQL, Redis, Docker, Kubernetes, and AWS for high-volume customer operations.
- Designed scalable **REST API** and **GraphQL** services with modular NestJS architecture, improving backend maintainability by **38%** and reducing duplicated business logic across service layers.
- Implemented a new payment reconciliation feature using **Node.js**, PostgreSQL stored procedures, Redis locks, and SQS events, reducing manual finance review work by **41%**.
- Resolved a production issue where duplicate webhook callbacks created inconsistent payment states by adding idempotency keys, transaction-safe updates, and structured retry policies.
- Migrated legacy **Express.js 4.x** services into **NestJS 10.x** modules with dependency injection, validation pipes, OpenAPI contracts, and centralized exception handling.
- Optimized slow reporting APIs by rewriting inefficient joins, adding indexed query paths, and caching calculated results in Redis, improving response time by **46%**.
- Created event-driven workflows with AWS Lambda, SNS, SQS, and Node.js workers to separate long-running jobs from user-facing APIs and improve system reliability.

- Improved authentication and authorization by implementing JWT rotation, RBAC policies, refresh-token storage, and audit logging across internal admin services.
- Fixed memory-leak issues in Node.js background workers by profiling heap snapshots, removing unbounded listeners, and controlling batch processing concurrency.
- Built reusable backend libraries for logging, validation, error handling, and API response standards, helping multiple teams ship features with fewer integration defects.
- Strengthened CI/CD pipelines using GitHub Actions, Docker images, automated Jest tests, and deployment checks, reducing release rollback incidents by **29%**.
- Collaborated with product, QA, DevOps, and client teams to translate complex business rules into secure backend features while keeping delivery flexible across sprint priorities.

VividCloud

Software Engineer | January 2016 – December 2020

- Developed cloud-based logistics, healthcare, and internal business platforms using **Node.js 10.x–14.x**, **JavaScript ES6+**, **Express.js 4.x**, MongoDB, PostgreSQL, Redis, and AWS.
- Built RESTful services for scheduling, order tracking, customer notifications, and reporting workflows, improving API reuse across web and mobile clients by **35%**.
- Implemented a real-time notification feature using Node.js workers, Redis queues, and WebSocket communication to keep users updated on status changes instantly.
- Fixed a recurring timeout issue in shipment APIs by separating synchronous processing from queue-based jobs and adding retry handling for third-party service failures.
- Migrated older callback-heavy Node.js modules into Promise-based and async/await patterns, making the codebase easier to test, debug, and extend.
- Improved MongoDB query performance by adding compound indexes, projection fields, and pagination controls, reducing average dashboard load time by **39%**.
- Created middleware for request validation, centralized error responses, request tracing, and secure header handling across multiple Express.js applications.
- Integrated AWS S3 for document uploads and secure file access, adding signed URLs, size validation, MIME checks, and cleanup jobs for expired records.
- Resolved a data mismatch bug between PostgreSQL and MongoDB services by introducing reconciliation scripts, audit fields, and scheduled consistency checks.
- Added Jest and Supertest coverage for critical APIs, increasing backend test coverage by **31%** and reducing regression defects during client releases.
- Worked flexibly across backend APIs, database tuning, frontend integration, and deployment troubleshooting whenever production priorities required cross-functional support.

Minimalist Moon

Software Developer | January 2013 – December 2015

- Built business web applications and API services using **Node.js 0.10.x–6.x**, **JavaScript**, **Express.js 3.x–4.x**, MySQL, MongoDB, jQuery, and early cloud deployment practices.
- Developed backend endpoints for customer management, inventory workflows, internal dashboards, and reporting tools used by small business clients.
- Implemented a new order-management module with Express.js routes, MySQL transactions, validation rules, and admin screens that reduced manual tracking errors by **28%**.
- Fixed session-loss issues caused by inconsistent cookie handling by standardizing Express session configuration, Redis-backed storage, and secure expiration rules.
- Migrated several older PHP-style server-rendered workflows into Node.js API endpoints while keeping existing frontend pages stable during gradual modernization.
- Improved MySQL performance by adding indexes, normalizing frequently updated tables, and reducing repeated queries inside high-traffic admin screens.

- Created reusable JavaScript utilities for form validation, date formatting, API calls, and error display to reduce repetitive frontend and backend code.
- Resolved file-upload failures by adding size limits, file-type checks, timeout handling, and clearer validation messages for non-technical users.
- Supported Linux server deployments with PM2, Nginx reverse proxy configuration, environment variables, and basic log review for production issue diagnosis.
- Worked across frontend, backend, database, and deployment tasks, applying **Node.js** skills flexibly in lean teams with fast-changing client requirements.

Microsoft

Software Developer Intern | *July 2012 – December 2012*

- Supported internal engineering tools using **JavaScript**, early **Node.js**, ASP.NET services, SQL Server, HTML, CSS, and automated test utilities.
- Assisted senior engineers with backend API maintenance, bug investigation, dashboard updates, and internal workflow improvements for engineering teams.
- Built small Node.js scripts to process JSON data, validate internal configuration files, and reduce repetitive manual checks during development activities.
- Fixed UI and API integration bugs by tracing request payloads, reviewing server logs, and improving validation messages for internal users.
- Helped migrate simple JavaScript utilities from older browser-specific patterns into cleaner modular code that was easier to maintain.
- Wrote SQL queries and lightweight reports to help teams inspect test results, service status, and workflow data more efficiently.
- Added unit-test coverage for shared JavaScript functions and supported regression testing before internal tool releases.
- Documented common setup issues, dependency errors, and debugging steps so new interns and junior developers could become productive faster.
- Collaborated with senior developers during code reviews and learned practical engineering habits around clean commits, readable code, and production-safe changes.

Education

Florida State University

Bachelor's Degree in Computer Science | 2008 – 2012